

Classification Algorithms: Placement and Project Guide

Shared setup and evaluation

All examples assume a supervised classification dataset. Use a stratified split, fit preprocessing **only on training data**, and report accuracy plus precision, recall, F1 and ROC-AUC (binary/probabilistic models). For imbalanced classes, prefer macro/weighted F1, PR-AUC, class weights, and a confusion matrix over raw accuracy.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.metrics import classification_report, roc_auc_score

X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=.2, stratify=y, random_state=42)

def evaluate(model):
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    print(classification_report(y_test, pred))
    if hasattr(model, "predict_proba"):
        print("ROC-AUC:", roc_auc_score(y_test, model.predict_proba(X_test)[: ,
1]))
```

Logistic Regression

1. Overview

Logistic regression is a linear probabilistic classifier. It is a strong, interpretable baseline for credit risk, medical triage, churn, and spam, and often remains a production choice when explanations and calibrated probabilities matter.

2. Intuition

It adds feature evidence into one score, then squashes that score into a probability. For example, income, repayment history, and debt each push a loan-default probability up or down.

3. Prerequisites

Linear algebra (dot product), probability, sigmoid, gradients, regularization, feature scaling, and train/test splits.

4. Core Concepts

- **Log-odds:** $w \cdot x + b$ models $\log(p/(1-p))$; coefficients have directional meaning. Interview: interpret $\exp(w_j)$ as an odds ratio for a one-unit increase.
- **Sigmoid:** maps any score to $[0, 1]$; a threshold converts probability to class. Interview: the threshold need not be 0.5.
- **Regularization:** L2 shrinks weights; L1 can select features. Interview: regularization reduces variance, not data leakage.

5. Algorithm / Working Process

Input is numeric/encoded features. Compute a linear score, sigmoid probability, and threshold it. Training minimizes log loss over labeled rows; inference returns a class and usually `predict_proba`.

6. Mathematical Foundation

$p(y=1|x) = \sigma(w^T x + b) = 1 / (1 + e^{-(w^T x + b)})$. Binary cross-entropy is $-\sum [y \log p + (1-y) \log(1-p)] + \lambda ||w||^2$ (L2). Gradient-based solvers find weights that lower this convex objective.

7. Practical Implementation

```
from sklearn.linear_model import LogisticRegression
model = make_pipeline(StandardScaler(), LogisticRegression(C=1.0, max_iter=2000))
evaluate(model)
```

8. Code Explanation

`StandardScaler` makes coefficient regularization fair across scales. `C` is inverse regularization strength: smaller `C` means more shrinkage. The pipeline prevents scaling leakage.

9. Training / Evaluation

Tune `C`, penalty, and class weights with stratified cross-validation. Use calibration curves when probabilities drive decisions. Inspect coefficient stability and validate a decision threshold against business cost.

10. Complexity and Cost

Training is roughly $O(n \cdot d \times \text{iterations})$; prediction is $O(d)$ per row. Memory is small; CPU is usually enough.

11. Common Use Cases

Default/churn prediction, click-through rate, disease screening, spam, and interpretable scorecards.

12. Common Mistakes

Scaling before split, treating coefficients as causal, retaining perfectly collinear features without regularization, using accuracy for imbalance, and trusting uncalibrated threshold choices.

13. Edge Cases / Limitations

It learns a linear boundary in feature space. Strong nonlinear interactions, complete separation, and high-cardinality sparse categories require feature engineering, regularization, or another model.

14. Variations

- **Multinomial logistic regression:** softmax for mutually exclusive classes; placement-essential.
- **One-vs-rest:** one binary model per class; useful for multilabel/many classes.
- **Elastic net:** L1+L2 for correlated high-dimensional features; useful in text/genomics.

15. Related Topics

Logistic regression and linear SVM both learn linear boundaries; logistic regression directly models probabilities. It is also a single-neuron neural network with sigmoid activation.

16. Interview Questions

Question	Answer
Why not linear regression for classes?	Predictions are unbounded and squared error is mismatched to Bernoulli labels.
What does a coefficient mean?	Change in log-odds, holding other features fixed.
Why scale?	Regularization and gradient solvers depend on scale.
L1 vs L2?	L1 induces sparsity; L2 smoothly shrinks correlated weights.
Is it nonlinear?	Only after nonlinear feature transforms.
Why log loss?	It is Bernoulli negative log-likelihood and penalizes confident errors.
What is C ?	Inverse regularization strength.
Can it do multiclass?	Yes, softmax/multinomial or one-vs-rest.
How change recall?	Lower the decision threshold after validation.
When prefer it?	Small/medium tabular data needing speed, explanations, probabilities.

17. Practice Tasks

Implement sigmoid and log loss in NumPy; build a churn classifier with class weights; compare L1/L2 coefficient sparsity; tune threshold for recall; diagnose a leakage-prone scaler.

18. Project Ideas

- **Loan risk scorecard:** scikit-learn, SHAP/coefficients, German Credit; strong interpretable-ML resume story.
- **Churn probability API:** FastAPI + logistic regression, Telco Churn; demonstrates deployment and calibration.
- **SMS spam detector:** TF-IDF + logistic regression, SMS Spam Collection; practical NLP baseline.

19. Quick Revision

Key idea: linear log-odds. Formula: $\sigma(\mathbf{w}^T \mathbf{x} + b)$. Use for interpretable probabilities. Metrics: ROC-AUC/F1. Trap: scale inside CV. One-liner: "A regularized linear classifier trained by log loss."

20. Final Cheat Sheet

Definition: probabilistic linear classifier. Input/output: features $\rightarrow$ class probabilities. Steps: score, sigmoid/softmax, optimize log loss. Hyperparameters: C , penalty, class weight. Pros: fast, interpretable; cons: linear boundary. Best: calibrated tabular baseline.

K-Nearest Neighbors

1. Overview

KNN is a non-parametric, instance-based classifier that labels a sample from nearby labeled examples. It is useful for small datasets, similarity search, and as an intuitive baseline.

2. Intuition

To identify a fruit, look at the k most similar known fruits and take a vote. Neighbors close in a meaningful feature space should share a label.

3. Prerequisites

Distance metrics, feature scaling, voting, curse of dimensionality, and basic cross-validation.

4. Core Concepts

- **Distance metric:** Euclidean is common; choice defines "similar." Interview: scaling is mandatory because distance is scale-sensitive.
- **k :** small k overfits noise; large k oversmooths. Interview: use odd k for binary ties.
- **Weighting:** distance weights give close neighbors more influence. Interview: it can improve local boundaries.

5. Algorithm / Working Process

Store training points. At inference, compute distance from query to every stored point, select k , vote (optionally weighted), and output the majority class/proportion.

6. Mathematical Foundation

Euclidean distance is $d(\mathbf{x}, \mathbf{z}) = \sqrt{\sum (\mathbf{x}_j - \mathbf{z}_j)^2}$. Estimated class probability is the fraction (or normalized weighted fraction) of the k neighbors in that class. There is no conventional parameter-fitting loss.

7. Practical Implementation

```
from sklearn.neighbors import KNeighborsClassifier
model = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=7,
weights="distance"))
evaluate(model)
```

8. Code Explanation

The scaler makes all dimensions comparable. `n_neighbors=7` controls locality; `weights="distance"` reduces the effect of far boundary neighbors.

9. Training / Evaluation

“Training” stores data. Tune `k`, metric, and weights by CV. Use stratification and scale numerical columns; consider PCA or feature selection in high dimensions.

10. Complexity and Cost

Training: $O(nd)$ storage. Brute-force inference: $O(nd)$ per query and memory $O(nd)$ —often expensive at scale. KD/Ball trees help only in low dimensions.

11. Common Use Cases

Handwritten digit baselines, recommendation-like similarity tasks, local anomaly checks, and small medical datasets.

12. Common Mistakes

Not scaling, choosing `k` on the test set, using Euclidean distance on one-hot high-cardinality data, ignoring ties/class imbalance, and deploying it where low latency matters.

13. Edge Cases / Limitations

Performance deteriorates in high dimensions; irrelevant features corrupt distance. It has poor extrapolation, costly inference, and can expose stored training examples.

14. Variations

- **Radius neighbors:** use all points within a radius; density-aware but may find none.
- **Weighted KNN:** closer points vote more; placement-important.
- **Approximate nearest neighbor:** indexes for large-scale retrieval; project/research relevant.

15. Related Topics

KNN is the classification counterpart of KNN regression and a conceptual foundation for vector retrieval. Unlike trees, its work happens primarily at inference.

16. Interview Questions

Question	Answer
Is KNN lazy learning?	Yes; it defers computation until prediction.
Why scale features?	A large-scale feature otherwise dominates distances.
Small vs large k ?	Small: low bias/high variance; large: higher bias/lower variance.
Training cost?	Mostly storing the data.
Prediction cost?	Brute force is linear in training rows.
What is curse of dimensionality?	Distances become less discriminative as dimensions grow.
How output probabilities?	Neighbor class proportions/weights.
Can it handle categorical data?	With a suitable metric/encoding, carefully.
How handle imbalance?	Weighted voting, resampling, or class-aware evaluation.
When avoid it?	Large, high-dimensional, or low-latency workloads.

17. Practice Tasks

Code Euclidean KNN; plot validation score against k ; compare raw versus scaled data; classify Iris; add feature selection to a text-like sparse dataset.

18. Project Ideas

- **Similar medical cases:** KNN + PCA, breast-cancer data; explain nearest examples.
- **Fashion image finder:** embeddings + nearest neighbors, Fashion-MNIST; demonstrates retrieval.
- **Local crop diagnosis:** phone-image embeddings + KNN, PlantVillage; simple incremental classifier.

19. Quick Revision

Key idea: nearby labels vote. Formula: Euclidean distance. Use: small, well-scaled low-dimensional data. Metrics: F1/accuracy. Trap: no scaling. One-liner: "Memory-based local classification."

20. Final Cheat Sheet

Definition: classifies by nearest stored observations. Input/output: vector $\rightarrow$ neighbor vote. Steps: distance, select k , vote. Hyperparameters: k , metric, weights. Pros: simple/nonlinear; cons: slow predictions. Best: similarity baselines.

Naive Bayes

1. Overview

Naive Bayes is a fast probabilistic classifier based on Bayes' theorem and conditional independence assumptions. It is especially effective for high-dimensional sparse text such as spam and document classification.

2. Intuition

For each class, ask: "How likely are these observed words/features if the message were spam?" Multiply evidence with the class prior, then choose the larger posterior.

3. Prerequisites

Conditional probability, Bayes' theorem, distributions (Gaussian/multinomial/Bernoulli), logarithms, and count vectorization.

4. Core Concepts

- **Bayes theorem:** $\text{posterior} \propto \text{likelihood} \times \text{prior}$. Interview: denominator is identical across candidate classes when choosing argmax .
- **Conditional independence:** features are assumed independent given class; unrealistic but useful. Interview: "naive" describes this assumption.
- **Smoothing:** Laplace smoothing prevents zero-probability unseen words. Interview: **alpha** controls smoothing.

5. Algorithm / Working Process

Estimate class priors and per-class feature distributions from labels. For a new row, add log prior and log likelihood of every feature for each class; choose the largest posterior score.

6. Mathematical Foundation

$P(y|x) \propto P(y) \prod_j P(x_j|y)$. Compute logs: $\log P(y|x) = \log P(y) + \sum_j \log P(x_j|y) + \text{constant}$. Multinomial NB estimates token probabilities from counts; Gaussian NB estimates mean/variance per feature/class.

7. Practical Implementation

```
from sklearn.naive_bayes import GaussianNB
from sklearn.pipeline import Pipeline
model = Pipeline([("scale", StandardScaler()), ("nb", GaussianNB())])
evaluate(model)
```

8. Code Explanation

GaussianNB assumes every continuous feature is Gaussian within each class. For word counts use **MultinomialNB(alpha=1.0)** with **CountVectorizer** or TF-IDF.

9. Training / Evaluation

Choose variant from data type, not popularity. Inspect calibration because NB probabilities can be overconfident. Tune smoothing; assess F1/PR-AUC for spam-like imbalance.

10. Complexity and Cost

Training and inference are approximately $O(nd)$ and $O(dC)$ respectively, with small memory. Excellent on CPU.

11. Common Use Cases

Spam filtering, news-topic classification, sentiment baselines, document routing, and simple diagnosis.

12. Common Mistakes

Using Gaussian NB on counts, omitting smoothing, multiplying tiny probabilities instead of logs, assuming independence must be literally true, and applying dense preprocessing to sparse text.

13. Edge Cases / Limitations

Correlated features double-count evidence; nonlinear feature interactions are missed. Poor probability calibration can hurt downstream decision systems.

14. Variations

- **Gaussian NB:** continuous features; placement-essential.
- **Multinomial NB:** counts/TF-IDF; essential for NLP.
- **Bernoulli NB:** binary feature occurrence; useful for short text.
- **Complement NB:** more robust to imbalanced text classes.

15. Related Topics

NB is a generative classifier: it models $P(\mathbf{x}|\mathbf{y})P(\mathbf{y})$, unlike logistic regression's direct $P(\mathbf{y}|\mathbf{x})$. LDA is also generative but jointly models correlated Gaussian features.

16. Interview Questions

Question	Answer
State Bayes theorem.	$P(\mathbf{y} \mathbf{x}) = \frac{P(\mathbf{x} \mathbf{y})P(\mathbf{y})}{P(\mathbf{x})}$
Why "naive"?	Conditional independence of features given class.
Why logs?	Avoid numerical underflow and turn products into sums.
What is smoothing?	Adds pseudo-counts to avoid zero likelihoods.
Gaussian vs multinomial?	Continuous Gaussian features vs count features.
Does independence need hold exactly?	No; useful classifiers can violate it.
What is a prior?	Class probability before observing features.
Why good for text?	Sparse high-dimensional counts fit its factorization and cost.
Is it discriminative?	No, it is generative.
Main weakness?	Correlation/interactions and often poor calibration.

17. Practice Tasks

Build spam classifier with TF-IDF + Multinomial NB; vary α ; compare Bernoulli and multinomial; compute posterior manually for two words; calibrate probabilities.

18. Project Ideas

- **Email spam filter:** TF-IDF + MultinomialNB, Enron/SMS; classic explainable NLP.
- **Ticket router:** CountVectorizer + ComplementNB, support tickets; operations automation.
- **News classifier:** 20 Newsgroups; compare NB to linear SVM.

19. Quick Revision

Key idea: prior times independent likelihoods. Formula: $P(y) \prod P(x_j | y)$. Use: text/count baseline. Metrics: F1/PR-AUC. Trap: wrong distribution. One-liner: "Fast generative classifier with a useful simplifying assumption."

20. Final Cheat Sheet

Definition: Bayes-rule classifier with conditional independence. Input/output: features $\rightarrow$ posterior/class. Steps: estimate priors/likelihoods, sum log scores. Hyperparameters: smoothing. Pros: fast, text-friendly; cons: correlated features. Best: sparse NLP.

Decision Tree

1. Overview

A decision tree recursively partitions feature space with if/then rules. It is interpretable, handles nonlinear effects and mixed scales, and is the building block for tree ensembles.

2. Intuition

Like a flowchart: "Is income > 50k? Is debt ratio < 0.4?" Each question creates purer groups until a leaf gives a class vote.

3. Prerequisites

Entropy/Gini impurity, conditional probability, recursion, overfitting, and cross-validation.

4. Core Concepts

- **Split:** feature threshold separates rows. Why: reduces label mixture. Interview: CART typically makes binary splits.
- **Impurity:** Gini $1 - \sum p_k^2$ or entropy $-\sum p_k \log p_k$; lower means purer node.
- **Pruning:** limits depth/leaves to control variance. Interview: unpruned trees usually overfit.

5. Algorithm / Working Process

At every node, test candidate feature thresholds, choose the largest impurity reduction, recurse on children, then predict the leaf's majority class/probability.

6. Mathematical Foundation

Information gain is $I(\text{parent}) - \text{weighted } I(\text{children})$. A leaf probability is the class fraction among its training rows. CART greedily optimizes each split, not the globally best tree.

7. Practical Implementation

```
from sklearn.tree import DecisionTreeClassifier
model = DecisionTreeClassifier(max_depth=4, min_samples_leaf=8, random_state=42)
evaluate(model)
```

8. Code Explanation

`max_depth` and `min_samples_leaf` are regularizers. A leaf minimum avoids rules learned from a handful of noisy rows; no scaling is needed.

9. Training / Evaluation

Tune depth, leaf size, split criterion, and `ccp_alpha` using CV. Visualize the tree and compare train versus validation scores; use time/group splits when rows are dependent.

10. Complexity and Cost

Typical training is about $O(nd \log n)$ per tree, depending on split search. Inference is $O(\text{depth})$, memory stores nodes.

11. Common Use Cases

Eligibility rules, risk triage, feature interaction discovery, explainable baselines, and preprocessing-free tabular classification.

12. Common Mistakes

Reading a deep tree as stable truth, failing to limit depth, using feature importance as causality, leaking future variables, and assuming scaling helps.

13. Edge Cases / Limitations

Trees are high variance: small data changes can change rules. Axis-aligned splits can need deep trees for diagonal boundaries and may favor high-cardinality features.

14. Variations

- **CART:** binary Gini/variance splitting; core placement topic.
- **Cost-complexity pruning:** penalizes tree size; useful for interpretability.
- **Oblique trees:** split on linear combinations; research-oriented.

15. Related Topics

Random Forest reduces tree variance through bagging; boosting corrects tree residual errors sequentially. A single tree is most interpretable but generally less accurate.

16. Interview Questions

Question	Answer
Gini vs entropy?	Both measure impurity; Gini is often slightly faster, outcomes are similar.
Why trees overfit?	Greedy splits can memorize small, pure leaves.
How prevent it?	Depth/leaf limits, pruning, CV.
Need scaling?	No; threshold comparisons preserve ordering.
What is information gain?	Parent impurity minus weighted child impurity.
Is splitting global optimum?	No, it is greedy.
How classify at leaf?	Majority class or class proportions.
Why unstable?	A tiny data change may alter early splits.
How handle missing values?	Depends on library; impute or use native support.
Why use a tree?	Transparent rules and nonlinear interactions.

17. Practice Tasks

Implement Gini; train depth 1–20 and plot CV score; render a shallow tree; compare pruning; find a leakage feature from suspiciously perfect accuracy.

18. Project Ideas

- **Loan approval rules:** sklearn tree + visualization, German Credit; easy stakeholder explanation.
- **Student performance triage:** UCI Student Performance; interpretable interventions.
- **Fraud rule explorer:** imbalanced synthetic/transaction data; demonstrate precision-recall trade-offs.

19. Quick Revision

Key idea: greedy if/then partitions. Formula: impurity reduction. Use: explainable nonlinear tabular data. Metrics: F1/AUC. Trap: unrestricted depth. One-liner: “A high-variance rule learner.”

20. Final Cheat Sheet

Definition: recursive partition classifier. Input/output: row → leaf probability/class. Steps: choose split, recurse, vote. Hyperparameters: depth, leaf size, pruning. Pros: readable/no scaling; cons: unstable. Best: rules and tree ensembles.

Random Forest

1. Overview

Random Forest is a bagged ensemble of decorrelated decision trees. It is a robust default for many tabular datasets and provides useful baseline feature importance.

2. Intuition

Ask many imperfect but diverse trees to vote. Individual trees overreact to samples; averaging their votes cancels much of that variance.

3. Prerequisites

Decision trees, bootstrap sampling, voting, bias-variance trade-off, and out-of-bag validation.

4. Core Concepts

- **Bagging:** train each tree on a sampled dataset; reduces variance. Interview: trees train independently.
- **Feature subsampling:** each split sees random features; decorrelates trees. Interview: this is crucial beyond bootstrap sampling.
- **OOB score:** rows omitted from a tree evaluate it; a cheap internal estimate.

5. Algorithm / Working Process

Draw a bootstrap sample per tree; grow a tree while restricting candidate features at each split; average probability/vote over all trees at inference.

6. Mathematical Foundation

Forest prediction is $\text{argmax}_k \frac{1}{T} \sum_t I(h_t(x)=k)$ or averaged probabilities. Averaging correlated estimators reduces variance less than averaging diverse ones, motivating feature randomness.

7. Practical Implementation

```
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier(n_estimators=400, min_samples_leaf=3, n_jobs=-1,
random_state=42)
evaluate(model)
```

8. Code Explanation

`n_estimators` increases stability; `min_samples_leaf` regularizes trees; `n_jobs=-1` uses CPU cores. Scaling is unnecessary.

9. Training / Evaluation

Tune `max_features`, leaf size, depth, and number of trees. Use OOB as a quick diagnostic, but retain a validation/test set. Prefer permutation importance for less biased importance estimates.

10. Complexity and Cost

Training is roughly number-of-trees times tree cost and parallelizes well. Inference traverses every tree; model memory can be substantial. GPU is unnecessary.

11. Common Use Cases

Credit scoring, tabular healthcare, transaction classification, remote sensing, and robust first-pass models.

12. Common Mistakes

Believing more trees cure leakage, leaving trees fully deep on tiny noisy data, interpreting impurity importance blindly, ignoring class imbalance, and tuning on test data.

13. Edge Cases / Limitations

Less compact/fast than a linear model, weak extrapolation, can be outperformed by boosting on structured data, and cannot natively capture ordered time without suitable features/splits.

14. Variations

- **Balanced Random Forest:** bootstraps balanced classes; useful for imbalance.
- **Extremely Randomized Trees:** random split thresholds; faster/more diverse.
- **Random subspace forests:** feature randomness at tree level; useful high-dimensional data.

15. Related Topics

It is bagging of trees; Extra Trees injects more split randomness. Gradient boosting builds trees sequentially and often wins accuracy but needs more tuning.

16. Interview Questions

Question	Answer
Why better than one tree?	Averaging diverse trees lowers variance.
What does bootstrap mean?	Sampling rows with replacement.
Why random features?	To decorrelate trees and improve averaging.
Does it overfit with more trees?	Usually performance plateaus; cost grows.
What is OOB?	Prediction using trees that did not train on that row.
Need scaling?	No.
How handle imbalance?	<code>class_weight</code> , balanced sampling, F1/PR-AUC.
Is it parallel?	Trees can train independently.
Feature-importance caveat?	Impurity importance favors many-split features.
RF vs boosting?	RF reduces variance; boosting sequentially reduces bias.

17. Practice Tasks

Compare OOB and test score; tune `max_features`; compare impurity/permutation importance; test class weights; benchmark 50 versus 500 trees.

18. Project Ideas

- **Customer churn benchmark:** RF + permutation importance, Telco Churn; strong tabular baseline.
- **Forest fire risk:** weather/terrain features, UCI Forest Fires variant; spatial feature engineering.
- **Transaction fraud classifier:** class weights + PR-AUC, IEEE-CIS/Kaggle subset; imbalance experience.

19. Quick Revision

Key idea: many decorrelated trees vote. Formula: average tree probabilities. Use: robust tabular baseline. Metrics: F1/AUC. Trap: biased feature importance. One-liner: "Bagging plus random features lowers tree variance."

20. Final Cheat Sheet

Definition: tree bagging ensemble. Input/output: features → averaged vote/probability. Steps: bootstrap, random splits, aggregate. Hyperparameters: trees, max features, leaf size. Pros: robust; cons: bulky/less interpretable. Best: general tabular classification.

Support Vector Machine

1. Overview

SVM finds a separating boundary with the largest margin between classes. Kernel SVMs model complex boundaries and work well on small-to-medium, high-dimensional datasets.

2. Intuition

Among many separating lines, choose the one leaving the widest safety corridor between classes. Only the closest points—the support vectors—determine that corridor.

3. Prerequisites

Vectors/dot products, optimization, margins, hinge loss, regularization, kernels, and feature scaling.

4. Core Concepts

- **Margin:** distance from boundary to closest points; larger generalizes better. Interview: support vectors lie on/inside margin.
- **Soft margin/C:** allows violations; high `C` fits harder, low `C` regularizes.
- **Kernel trick:** replace dot product with similarity function without explicitly mapping features. `gamma` controls RBF locality.

5. Algorithm / Working Process

Scale input; select linear/RBF kernel; optimize a maximum-margin boundary with penalties for violations. Inference scores similarity to support vectors, then uses sign/class; probability requires calibration.

6. Mathematical Foundation

Hard margin minimizes $\frac{1}{2} \|w\|^2$ subject to $y_i(w^T x_i + b) \geq 1$. Soft margin adds $C \sum \xi_i$; equivalent hinge-loss objective $\frac{1}{2} \|w\|^2 + C \sum \max(0, 1 - y_i f(x_i))$. RBF kernel: $K(x, z) = \exp(-\gamma \|x - z\|^2)$.

7. Practical Implementation

```
from sklearn.svm import SVC
model = make_pipeline(StandardScaler(), SVC(C=1.0, kernel="rbf", gamma="scale",
probability=True))
evaluate(model)
```

8. Code Explanation

Scaling is essential for RBF distances. `C` balances margin and training errors; `gamma="scale"` is a sensible initial width. `probability=True` adds costly probability calibration.

9. Training / Evaluation

Tune `C`, `gamma`, and kernel via logarithmic CV search. Use `LinearSVC`/SGD for very large sparse data. Calibrate probability outputs if they affect ranking/thresholding.

10. Complexity and Cost

Kernel SVM training can be roughly quadratic or worse in rows and memory; inference depends on support-vector count. Linear SVM is far cheaper. CPU is typical.

11. Common Use Cases

Text classification with linear kernels, bioinformatics, small image-feature datasets, and margin-sensitive binary classification.

12. Common Mistakes

Skipping scaling, setting huge `C`/gamma, using RBF SVC on millions of rows, interpreting SVC scores as calibrated probabilities, and forgetting multiclass strategy cost.

13. Edge Cases / Limitations

Kernel SVMs do not scale well to large `n`, are sensitive to feature scale/hyperparameters, and are less interpretable than linear models.

14. Variations

- **Linear SVM:** linear kernel, scalable; placement-essential.

- **RBF SVM:** flexible nonlinear boundary; common projects.
- **Polynomial kernel:** interaction-like geometry; less common.
- **One-class SVM:** novelty detection; advanced extension.

15. Related Topics

Logistic regression uses log loss and gives native probabilities; linear SVM uses hinge loss and margin. Kernel methods connect SVMs to Gaussian processes.

16. Interview Questions

Question	Answer
What are support vectors?	Training points that determine the boundary/margin.
What does maximum margin buy?	Lower capacity and better generalization intuition.
What does C do?	Penalizes violations; high C fits training points harder.
What does gamma do?	RBF influence radius; high gamma makes very local regions.
Why scale?	Margin and kernels use feature magnitudes/distances.
Kernel trick?	Compute high-dimensional inner products implicitly.
Is SVM probabilistic?	Not inherently; calibrate scores.
Hinge vs log loss?	Hinge focuses on margin; log loss models probabilities.
Why not big data RBF?	Kernel matrix/optimization scale poorly.
Linear vs RBF?	Use linear for near-linear/high-dimensional large data; RBF for smaller nonlinear data.

17. Practice Tasks

Plot C/γ decision surfaces; compare linear and RBF on text/PCA data; count support vectors; calibrate an SVM; demonstrate scaling failure.

18. Project Ideas

- **Cancer diagnosis:** RBF SVM, Wisconsin data; explain tuning and recall.
- **Document classifier:** TF-IDF + LinearSVC, 20 Newsgroups; scalable NLP.
- **Face-expression classifier:** HOG + SVM, FER subset; classic computer vision pipeline.

19. Quick Revision

Key idea: widest separating margin. Formula: hinge loss. Use: small/medium high-dimensional data. Metrics: F1/AUC. Trap: scale and tune C/gamma. One-liner: "A margin maximizer defined by support vectors."

20. Final Cheat Sheet

Definition: maximum-margin classifier. Input/output: vector $\rightarrow$ signed score/class. Steps: scale, kernel similarity, optimize hinge loss. Hyperparameters: C, gamma, kernel. Pros: powerful boundaries; cons: costly kernels. Best: nonlinear medium-size data or linear text.

Gradient Boosting

1. Overview

Gradient boosting builds weak learners sequentially, each correcting the previous ensemble's errors. It is a powerful foundation for modern tabular classification.

2. Intuition

Start with a crude prediction. Train a small tree to focus on what it gets wrong, add it, and repeat. Many modest corrections form a strong model.

3. Prerequisites

Decision trees, loss functions, gradients/residuals, learning rate, regularization, and bias-variance trade-off.

4. Core Concepts

- **Additive model:** $F_M = F_{M-1} + \eta h_M$; why: incremental error correction. Interview: learners are sequential, unlike RF.
- **Negative gradient:** target for next learner; for squared error it is residual. Interview: boosting generalizes residual fitting to arbitrary differentiable loss.
- **Shrinkage:** small learning rate needs more trees but regularizes.

5. Algorithm / Working Process

Initialize class score, compute loss gradient per row, fit a shallow tree to that gradient, add scaled tree, repeat; convert final scores to probabilities/classes.

6. Mathematical Foundation

Minimize $\sum L(y_i, F(x_i))$. At step m , fit h_m to $-\partial L / \partial F$; update $F_m = F_{m-1} + \eta h_m$. Binary logistic loss is cross-entropy on a logit score.

7. Practical Implementation

```
from sklearn.ensemble import GradientBoostingClassifier
model = GradientBoostingClassifier(n_estimators=200, learning_rate=.05,
max_depth=2, random_state=42)
evaluate(model)
```

8. Code Explanation

Shallow trees are weak learners. The learning rate/tree-count pair controls capacity; lower rate plus more estimators is usually safer than a few aggressive trees.

9. Training / Evaluation

Use a validation set/early stopping where available. Tune learning rate, estimators, depth, subsampling, and leaf limits. Watch train-validation gap; use cross-validation for modest data.

10. Complexity and Cost

Trees train sequentially, so parallelism is limited across boosting rounds. Cost is approximately trees times tree training; inference traverses all trees.

11. Common Use Cases

Structured risk models, ranking, insurance pricing, fraud, and competition-grade tabular baselines.

12. Common Mistakes

Too many/deep trees, high learning rate, no validation early stopping, using it as a cure for leakage, and interpreting gain importance causally.

13. Edge Cases / Limitations

Sensitive to noisy labels/outliers and hyperparameters; native sklearn version lacks some optimized categorical/missing-value features offered by modern libraries.

14. Variations

- **Stochastic gradient boosting:** subsamples rows; reduces variance.
- **Histogram gradient boosting:** bins features for speed; sklearn's scalable option.
- **XGBoost/LightGBM/CatBoost:** optimized, regularized descendants; placement-essential.

15. Related Topics

Random Forest averages independent trees to reduce variance; gradient boosting sequentially reduces bias. AdaBoost is a historical boosting method that reweights examples.

16. Interview Questions

Question	Answer
Bagging vs boosting?	Bagging trains independently to reduce variance; boosting trains sequentially to reduce bias.
What fits next tree?	Negative loss gradient (residual for squared loss).
Why shallow trees?	They are weak learners and regularize additive fitting.
Learning-rate trade-off?	Smaller rate needs more trees and often generalizes better.

Question	Answer
Does it parallelize by trees?	Not fully; rounds depend on prior rounds.
Why early stopping?	Stops before validation loss degrades.
What causes overfit?	Deep trees, too many rounds, high rate, noisy data.
Stochastic boosting?	Row subsampling per round.
Is it a gradient method?	Functional gradient descent in prediction space.
Why strong on tabular data?	Captures nonlinear interactions with engineered regularization.

17. Practice Tasks

Compare RF versus boosting; plot validation score by estimators; tune learning-rate/trees jointly; add subsampling; inspect early stopping behavior.

18. Project Ideas

- **Insurance claim classifier:** GradientBoosting, Kaggle insurance data; explain cost-sensitive thresholding.
- **Customer propensity model:** tabular marketing dataset; compare RF and boosting.
- **Feature-ablation lab:** benchmark linear/tree/boosting on UCI Adult; research-style analysis.

19. Quick Revision

Key idea: sequential error correction. Formula: $F_m = F_{m-1} + \eta h_m$. Use: strong tabular model. Metrics: AUC/F1. Trap: excessive depth/rate. One-liner: "Functional gradient descent using trees."

20. Final Cheat Sheet

Definition: sequential weak-tree ensemble. Input/output: features $\rightarrow$ summed score/probability. Steps: gradient, tree, scaled update. Hyperparameters: rate, trees, depth, subsample. Pros: accurate; cons: tuning/sequential cost. Best: tabular prediction.

XGBoost

1. Overview

XGBoost is an optimized, regularized gradient-tree boosting library. It is a standard choice for tabular competitions and production models because of strong accuracy, missing-value handling, and mature tooling.

2. Intuition

It is gradient boosting with more discipline: each new tree corrects errors, while explicit penalties discourage overly complex trees and efficient engineering makes the process fast.

3. Prerequisites

Gradient boosting, decision trees, derivatives, regularization, cross-validation, and categorical feature encoding.

4. Core Concepts

- **Second-order objective:** uses gradient and Hessian; gives better split approximation. Interview: it optimizes a Taylor expansion of loss.
- **Regularization:** penalizes leaf count and leaf weights; limits complexity.
- **Column/row sampling:** adds diversity and speed; reduces overfit.

5. Algorithm / Working Process

For each round, compute first/second loss derivatives, score candidate splits using their aggregate derivatives, grow a regularized tree, then add its weighted output to current logits. Missing values follow learned default split directions.

6. Mathematical Foundation

Objective: $\sum l(y_i, \hat{y}_i) + \sum \Omega(f_t)$, commonly $\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum w_j^2$. For a leaf with gradient sum G and Hessian sum H , optimal weight is approximately $-G/(H+\lambda)$; split gain compares child versus parent scores.

7. Practical Implementation

```
# pip install xgboost
from xgboost import XGBClassifier
model = XGBClassifier(n_estimators=500, learning_rate=.03, max_depth=4,
                      subsample=.8, colsample_bytree=.8, eval_metric="logloss",
                      random_state=42)
evaluate(model)
```

8. Code Explanation

Use low `learning_rate` with many estimators. `subsample` and `colsample_bytree` regularize; `eval_metric` makes optimization reporting explicit. In real work add an evaluation set and early stopping.

9. Training / Evaluation

Tune depth/min-child-weight, learning rate/estimators, sampling, `reg_lambda`, and `reg_alpha`. Use early stopping on validation data and native categorical support only when version/data pipeline supports it; otherwise encode safely.

10. Complexity and Cost

More optimized than basic boosting; histogram/parallel split search helps. Training remains sequential by rounds; memory and CPU/GPU usage grow with rows, bins, and trees.

11. Common Use Cases

Credit risk, fraud, click prediction, ranking, pricing, and Kaggle-style tabular problems.

12. Common Mistakes

Using test set for early stopping, setting both deep trees and high rate, ignoring imbalance (`scale_pos_weight`), one-hot exploding cardinal categories, and skipping reproducible seeds.

13. Edge Cases / Limitations

Can overfit noisy labels, needs tuning, and may be excessive for a simple linear relationship. Tree outputs do not extrapolate smoothly beyond training patterns.

14. Variations

- **gbtree**: default tree booster; essential.
- **gblinear**: linear booster; use for sparse linear relationships.
- **DART**: dropout trees; can regularize but is slower/less routine.

15. Related Topics

XGBoost, LightGBM, and CatBoost are gradient boosting libraries. LightGBM prioritizes histogram/leaf-wise speed; CatBoost excels with categorical features.

16. Interview Questions

Question	Answer
XGBoost over classic GBM?	Regularization, second-order splits, optimized/parallel implementation, missing handling.
What is Hessian?	Second derivative of loss used in split/leaf approximation.
What is gamma?	Minimum loss reduction required to split.
Lambda/alpha?	L2/L1 regularization on leaf weights.
Why early stop?	Selects tree count by validation performance.
Handle imbalance?	Weights, PR metrics, threshold, resampling.
Why low learning rate?	Smaller safer updates, usually with more trees.
Missing values?	Learns a default direction per split.
Feature importance caveat?	Gain/count are not causal and can be biased.
XGBoost vs RF?	Sequential bias reduction versus independent variance reduction.

17. Practice Tasks

Train with early stopping; tune `max_depth` and `min_child_weight`; compare class weights; inspect SHAP/permutation importance; compare CPU and histogram methods.

18. Project Ideas

- **Default probability model:** XGBoost + calibration, Home Credit subset; high resume relevance.
- **Fraud ranking system:** XGBoost + PR-AUC, credit-card fraud; shows imbalance handling.
- **Demand classification:** tabular retail features; showcase time-safe validation.

19. Quick Revision

Key idea: regularized second-order tree boosting. Formula: $\text{loss} + \gamma T + \lambda \Sigma w^2 / 2$. Use: high-performing tabular. Metrics: AUC/PR-AUC/F1. Trap: validation leakage. One-liner: "Industrial-strength regularized gradient boosting."

20. Final Cheat Sheet

Definition: optimized gradient-tree ensemble. Input/output: table $\rightarrow$ logits/probabilities. Steps: derivatives, split gain, additive trees. Hyperparameters: rate, trees, depth, child weight, subsample, regularization. Pros: strong/fast; cons: tuning. Best: tabular benchmarks.

LightGBM

1. Overview

LightGBM is a histogram-based gradient boosting library designed for high speed and large datasets. It often trains faster than level-wise boosting on large tabular data.

2. Intuition

Instead of checking every exact threshold, bucket feature values into bins. Grow the currently most promising leaf first, spending model capacity where error reduction is largest.

3. Prerequisites

Gradient boosting, histograms/bins, tree growth strategies, regularization, and categorical encoding.

4. Core Concepts

- **Histogram splitting:** discretizes values; faster, lower memory. Interview: bins approximate split thresholds.
- **Leaf-wise growth:** split the leaf with best gain, not every level; accurate but can overfit.
- **GOSS/EFB:** gradient-based row sampling and exclusive-feature bundling; advanced interview topics for speed.

5. Algorithm / Working Process

Bin features, calculate gradient statistics, choose the best leaf/split by gain, grow leaf-wise under leaf/depth constraints, and add trees sequentially. It supports categorical features via specialized splits when supplied correctly.

6. Mathematical Foundation

It minimizes the same regularized boosting objective as XGBoost but aggregates gradients/Hessians per histogram bin. Leaf-wise selection greedily chooses the split with greatest estimated loss reduction globally.

7. Practical Implementation

```
# pip install lightgbm
from lightgbm import LGBMClassifier
model = LGBMClassifier(n_estimators=500, learning_rate=.03, num_leaves=31,
                      subsample=.8, colsample_bytree=.8, random_state=42)
evaluate(model)
```

8. Code Explanation

`num_leaves` is the central complexity control for leaf-wise trees. Keep learning rate low and use validation callbacks/early stopping in production training.

9. Training / Evaluation

Tune `num_leaves`, `min_child_samples`, `max_depth`, learning rate, feature/bagging fractions. Guard against overfit with leaf limits and early stopping; use time-aware splits for events.

10. Complexity and Cost

Histogram bins reduce split-search cost and memory. It is CPU-efficient and GPU-capable; leaf-wise growth can create deeper, more costly individual paths.

11. Common Use Cases

Large clickstream datasets, ranking, ad-tech, credit models, and high-row-count tabular systems.

12. Common Mistakes

Unbounded leaf-wise trees, treating integer category IDs as ordered numbers, using too many bins needlessly, test-set early stopping, and defaulting to accuracy on imbalance.

13. Edge Cases / Limitations

Small datasets can overfit; categorical handling still requires correct dtypes/validation. Leaf-wise trees can be less interpretable and reproducibility may need controlled threads/seeds.

14. Variations

- **DART booster:** dropout boosting; use when standard boosting overfits.

- **GOSS:** samples by gradient magnitude; important concept for large data.
- **Ranker:** ranking objective; valuable recommender/search extension.

15. Related Topics

Compared with XGBoost it uses histogram and leaf-wise growth; CatBoost often needs less categorical preprocessing. All are gradient-boosted decision trees.

16. Interview Questions

Question	Answer
Leaf-wise vs level-wise?	Leaf-wise splits best-gain leaf; level-wise expands a depth level uniformly.
Why histogram bins?	Faster approximate split search with lower memory.
Main overfit control?	<code>num_leaves</code> , minimum child samples, depth, early stopping.
What is GOSS?	Keeps high-gradient rows and samples low-gradient rows.
What is EFB?	Bundles rarely co-occurring sparse features.
Why fast?	Histograms, efficient implementation, sampling/bundling.
Handle categories?	Mark/use categorical features correctly; do not impose fake order.
LightGBM vs XGBoost?	Often faster/leaf-wise; XGBoost is level-wise by default and mature.
Why small learning rate?	Regularizes additive updates.
When avoid?	Very small/noisy data or when simple interpretable model suffices.

17. Practice Tasks

Benchmark LightGBM and XGBoost; tune leaves versus depth; compare categorical encoding methods; use early stopping; test GOSS on a large dataset.

18. Project Ideas

- **Click-through prediction:** LightGBM, Criteo sample; large-scale tabular story.
- **Loan-default model:** native categories + calibration, LendingClub subset.
- **Search ranker:** LGBMRanker, Microsoft LETOR; advanced recommendation/search project.

19. Quick Revision

Key idea: fast histogram, leaf-wise boosting. Formula: regularized gradient objective. Use: large tabular data. Metrics: AUC/NDCG/F1. Trap: too many leaves. One-liner: "A fast leaf-wise histogram GBDT."

20. Final Cheat Sheet

Definition: efficient gradient-boosted trees. Input/output: table → scores/classes. Steps: bin, gradients, best-leaf split, add tree. Hyperparameters: leaves, child samples, rate, trees. Pros: fast; cons: overfit risk. Best: high-volume tables.

CatBoost

1. Overview

CatBoost is gradient boosting designed to handle categorical variables and reduce target leakage through ordered statistics. It is an excellent tabular option when categories are prominent.

2. Intuition

Rather than naïvely replacing a category with its full-dataset target average, it encodes each row using only earlier rows in a random ordering—like predicting without peeking at that row's answer.

3. Prerequisites

Gradient boosting, categorical variables, target encoding leakage, ordered statistics, cross-validation, and log loss.

4. Core Concepts

- **Ordered target statistics:** category encodings use prior rows only; prevents target leakage. Interview: random permutations make this usable for training.
- **Ordered boosting:** reduces prediction shift between train and inference.
- **Symmetric trees:** same split condition per depth; efficient and regularized.

5. Algorithm / Working Process

Pass categorical column indices; CatBoost creates ordered category statistics under permutations, grows symmetric trees using gradients, and aggregates tree scores into probabilities.

6. Mathematical Foundation

For category value c , an ordered encoding resembles $(\sum \text{prior targets for } c + a \cdot \text{prior}) / (\text{count prior } c + a)$. The ensemble still minimizes regularized classification loss through boosting; order avoids using a row's own label in its encoding.

7. Practical Implementation

```
# pip install catboost
from catboost import CatBoostClassifier
model = CatBoostClassifier(iterations=500, depth=6, learning_rate=.05,
                           loss_function="Logloss", verbose=False, random_seed=42)
evaluate(model) # numeric demo; use cat_features=[...] for DataFrame categories
```

8. Code Explanation

`iterations` is tree count, `depth` controls symmetric-tree complexity, and `verbose=False` quiets logs. With a pandas DataFrame, pass `cat_features` and avoid manually target-encoding categories.

9. Training / Evaluation

Use an evaluation set and `early_stopping_rounds`. Tune iterations/rate/depth and `l2_leaf_reg`; validate categories unseen in training and use stratified/time splits appropriate to data generation.

10. Complexity and Cost

Efficient CPU/GPU training; categorical processing adds cost but may replace wide one-hot matrices. Inference is an ensemble traversal.

11. Common Use Cases

Retail/customer data, fraud, ads, pricing, and any table with many string/category columns.

12. Common Mistakes

Manually target-encoding with leakage, converting categories to arbitrary integers without declaring them categorical, using test data in category statistics, and ignoring unknown category behavior.

13. Edge Cases / Limitations

For purely numeric simple data it may offer little benefit over alternatives. Very high-cardinality identifiers can still encode leakage or memorization; validate by entity/time.

14. Variations

- **Ordered boosting:** default leakage-resistant concept; placement-important.
- **Plain boosting:** faster on some large/GPU workloads.
- **CatBoost ranking:** use for ranking/recommendation tasks.

15. Related Topics

CatBoost competes with XGBoost/LightGBM. Its key distinction is ordered treatment of categorical features, whereas naive target encoding must be out-of-fold.

16. Interview Questions

Question	Answer
Why CatBoost?	Strong boosting with native categorical handling and reduced target leakage.
What is target leakage in encoding?	A row's label influences its own feature representation.
How does ordered encoding help?	It uses only preceding rows in a permutation.
What are symmetric trees?	Each depth uses one shared split condition across nodes.
Need one-hot encoding?	Usually no; provide categorical features natively.

Question	Answer
Can category IDs be numeric?	Yes, but explicitly mark them categorical.
Main tuning knobs?	Iterations, learning rate, depth, L2, early stopping.
CatBoost vs LightGBM?	CatBoost is often easier/safer for categories; LightGBM may be faster on large numeric data.
Can identifiers leak?	Yes; entity/time validation is essential.
Why calibration check?	Boosting probabilities need not reflect real event frequencies.

17. Practice Tasks

Compare CatBoost to one-hot LightGBM; intentionally demonstrate leaky target encoding; evaluate unseen categories; tune depth; use SHAP for explanations.

18. Project Ideas

- **Retail churn:** CatBoost on demographics/products, Telco/retail dataset; highlights categories.
- **Used-car price class:** categorical make/model plus numeric features, CarDekho; realistic feature mix.
- **Fraud propensity:** merchant/category data, IEEE-CIS subset; leakage-aware validation.

19. Quick Revision

Key idea: categorical-aware ordered boosting. Formula: prior-only category mean. Use: category-heavy tables. Metrics: AUC/F1. Trap: IDs and split leakage. One-liner: "GBDT that learns categories without naïve target leakage."

20. Final Cheat Sheet

Definition: ordered categorical gradient boosting. Input/output: mixed table → probability/class. Steps: ordered stats, symmetric trees, boost. Hyperparameters: iterations, rate, depth, L2. Pros: categories; cons: tuning/cost. Best: heterogeneous tabular data.

Extra Trees

1. Overview

Extra Trees (Extremely Randomized Trees) is a tree ensemble that adds random split thresholds as well as random feature selection. This can make training fast and reduce variance.

2. Intuition

Instead of carefully searching every possible cut, propose random cuts and retain the best among them. Many highly randomized trees vote, trading a little bias for diversity.

3. Prerequisites

Decision trees, random forests, bias-variance trade-off, bagging, and impurity.

4. Core Concepts

- **Random thresholds:** threshold candidates are random, lowering split-search cost and correlation.
- **Whole-sample default:** sklearn Extra Trees generally uses the full sample unless `bootstrap=True`; interview contrast with RF.
- **Ensemble averaging:** reduces variance from random trees.

5. Algorithm / Working Process

At each node choose random features, generate random candidate thresholds, select best impurity-reducing random split, grow many trees, then average/vote predictions.

6. Mathematical Foundation

It uses the same Gini/entropy impurity reduction as trees, but candidate split set is randomized. Ensemble probability is average tree probability.

7. Practical Implementation

```
from sklearn.ensemble import ExtraTreesClassifier
model = ExtraTreesClassifier(n_estimators=400, min_samples_leaf=3, n_jobs=-1,
random_state=42)
evaluate(model)
```

8. Code Explanation

The API resembles Random Forest. More trees stabilize the deliberately random process; leaf-size regularization avoids overly specific rules.

9. Training / Evaluation

Tune trees, `max_features`, depth, and leaf size with CV. Compare against RF on validation, not assumptions; use permutation importance for interpretation.

10. Complexity and Cost

Often faster to train than RF because it avoids exhaustive threshold optimization. Inference and memory are similar forest costs.

11. Common Use Cases

Fast tabular baseline, high-dimensional noisy features, feature screening, and RF alternative.

12. Common Mistakes

Assuming it always beats RF, confusing threshold randomness with bootstrap sampling, ignoring seeds, and trusting impurity importance.

13. Edge Cases / Limitations

Random thresholds can add harmful bias on small data or when precise thresholds matter. Like forests, it is bulky and weak at extrapolation.

14. Variations

- **Extra Trees with bootstrap:** adds row sampling; compare with RF.
- **Randomized forest regressors:** continuous outcome counterpart.
- **Rotation forests:** random feature transformations; research-oriented diversity.

15. Related Topics

RF searches optimal thresholds over random feature subsets; Extra Trees randomizes thresholds too. Both are bagging-style variance reducers.

16. Interview Questions

Question	Answer
Extra Trees vs RF?	Extra Trees use random thresholds and often full data; RF finds best thresholds on bootstrap samples.
Why randomize thresholds?	Faster splits and more decorrelated trees.
Bias/variance effect?	Usually higher bias, lower variance/correlation.
Need scaling?	No.
How aggregate?	Vote or average probabilities.
Is it boosting?	No, trees are independently built.
Why more trees?	Stabilize randomness.
Can it overfit?	Yes; tune leaf/depth despite averaging.
Importance caveat?	Split-based importance can be biased.
When choose it?	Fast diversified forest baseline.

17. Practice Tasks

Benchmark Extra Trees/RF runtime and F1; change `max_features`; compare bootstrap setting; assess seed variability; evaluate permutation importance.

18. Project Ideas

- **Fast tabular benchmark:** compare ET/RF/GBDT on Adult Income; demonstrates experimental rigor.

- **Sensor fault classifier:** noisy UCI sensor data; emphasizes robustness.
- **Feature-screening report:** ET + permutation importance, telecom data; stakeholder-ready analysis.

19. Quick Revision

Key idea: forest with random thresholds. Formula: averaged tree votes. Use: fast noisy-tabular baseline.

Metrics: F1/AUC. Trap: confusing it with RF. One-liner: "More randomized bagged trees."

20. Final Cheat Sheet

Definition: extremely randomized tree ensemble. Input/output: table → averaged class probability. Steps: random features/thresholds, vote. Hyperparameters: trees, features, leaves. Pros: fast/diverse; cons: bias/bulk. Best: forest comparison baseline.

AdaBoost

1. Overview

AdaBoost (Adaptive Boosting) sequentially combines weak classifiers, commonly decision stumps, by focusing later learners on previously misclassified examples. It is historically important and still useful as a compact baseline on clean data.

2. Intuition

Each hard-to-classify training example gets more attention in the next round. Learners that perform well earn a larger vote in the final committee.

3. Prerequisites

Binary classification, decision stumps, weighted samples, exponential loss, ensembles, and label noise.

4. Core Concepts

- **Sample weights:** misclassified rows are upweighted; why: next learner concentrates on hard cases. Interview: this makes AdaBoost noise-sensitive.
- **Weak learner:** slightly better than chance is enough; stumps are common.
- **Learner weight:** accurate learner gets higher α ; poor learner gets little influence.

5. Algorithm / Working Process

Initialize equal sample weights; fit a weak classifier; compute weighted error; calculate learner weight; increase weights of mistakes and normalize; repeat; take a weighted class vote at inference.

6. Mathematical Foundation

For binary labels $y \in \{-1, +1\}$, $F(x) = \sum \alpha_t h_t(x)$, with $\alpha_t = \frac{1}{2} \log\left(\frac{1 - e_t}{e_t}\right)$. Update $w_i \leftarrow w_i \exp(-\alpha_t y_i h_t(x_i))$, then normalize. It minimizes exponential loss $\sum \exp(-y_i F(x_i))$.

7. Practical Implementation

```
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
model = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(max_depth=1, random_state=42),
    n_estimators=200, learning_rate=.5, random_state=42)
evaluate(model)
```

8. Code Explanation

The depth-1 tree is a decision stump. `n_estimators` is number of sequential weak learners; lower `learning_rate` shrinks their contribution. Older sklearn versions call `estimator base_estimator`.

9. Training / Evaluation

Tune stump depth, estimator count, and rate with CV. Plot validation error by rounds; investigate mislabeled/outlier rows if performance degrades as rounds grow.

10. Complexity and Cost

Sequential training costs roughly number-of-estimators times weak-tree cost; inference evaluates all weak learners. Memory is modest for stumps.

11. Common Use Cases

Clean tabular baselines, face detection historically (Viola–Jones), educational ensemble demonstrations, and simple rule ensembles.

12. Common Mistakes

Using deep base trees without validation, expecting robustness to label noise, ignoring sample-weight behavior, and claiming it fits residuals exactly like gradient boosting.

13. Edge Cases / Limitations

Outliers and incorrect labels repeatedly gain weight, causing overfit. Modern gradient-boosting libraries often outperform it on complex tabular problems.

14. Variations

- **SAMME/SAMME.R:** multiclass algorithms; placement-relevant.
- **Real AdaBoost:** uses class probabilities rather than hard votes.
- **Gentle/LogitBoost:** less aggressive alternatives; research/historical context.

15. Related Topics

AdaBoost reweights samples using exponential loss; gradient boosting fits negative gradients of a chosen loss. Random Forest trains trees independently rather than adaptively.

16. Interview Questions

Question	Answer
What is AdaBoost's key idea?	Focus subsequent weak learners on prior mistakes.
Why stumps?	They are weak, cheap, and reduce overfit versus deep learners.
How are mistakes treated?	Their sample weights increase.
What is alpha?	The vote weight based on weighted learner error.
Loss function?	Exponential loss.
Why noise-sensitive?	Noisy points get repeatedly upweighted.
Is training parallel?	No; rounds are sequential.
AdaBoost vs GBM?	Reweighting/exponential loss versus generic gradient loss fitting.
Can it multiclass?	Yes, e.g. SAMME.
When use it today?	Simple clean-data baseline or teaching, not default GBDT replacement.

17. Practice Tasks

Implement one weight update; compare stumps/depth-3 trees; inject label noise and graph degradation; compare AdaBoost with GradientBoosting; inspect sample weights.

18. Project Ideas

- **Face/non-face classifier:** Haar-like features + AdaBoost, OpenCV-style data; historical CV relevance.
- **Loan rule ensemble:** stumps on German Credit; highly explainable baseline.
- **Noise robustness experiment:** synthetic data; research-internship-style comparison.

19. Quick Revision

Key idea: boost weights on mistakes. Formula: $\alpha = .5 \log((1-e)/e)$. Use: clean simple data. Metrics: F1/AUC. Trap: label noise. One-liner: "Sequential weighted voting of weak learners."

20. Final Cheat Sheet

Definition: adaptive weak-learner ensemble. Input/output: row $\rightarrow$ weighted vote. Steps: fit, error, reweight, repeat. Hyperparameters: estimators, rate, base depth. Pros: simple; cons: noise-sensitive. Best: educational/clean-data baseline.

Linear Discriminant Analysis

1. Overview

LDA is a generative classifier that assumes every class is Gaussian with its own mean but a **shared covariance matrix**. It is fast, interpretable, and also used for supervised dimensionality reduction.

2. Intuition

Model each class as an ellipse of the same shape but centered differently. A point belongs to the class whose ellipse assigns it the highest probability; equal shape makes boundaries straight lines.

3. Prerequisites

Multivariate Gaussian distribution, mean/covariance, Bayes theorem, matrix inverse, linear algebra, and class priors.

4. Core Concepts

- **Shared covariance:** all classes share Σ ; why: yields linear boundaries and fewer parameters. Interview: this is LDA's defining assumption.
- **Discriminant score:** combines prior, mean, covariance, and sample; select largest score.
- **Projection LDA:** finds directions maximizing between-class over within-class variance; dimensions $\leq$ classes - 1.

5. Algorithm / Working Process

Estimate a mean per class, pooled covariance, and class prior from training data. For a query, compute every Gaussian discriminant score and choose max; optionally project data into discriminative lower dimensions.

6. Mathematical Foundation

$\delta_k(x) = x^T \Sigma^{-1} \mu_k - \frac{1}{2} \mu_k^T \Sigma^{-1} \mu_k + \log \pi_k$. Predict $\text{argmax}_k \delta_k(x)$. Under Gaussian class-conditionals with shared Σ , Bayes' rule makes log posterior linear in x .

7. Practical Implementation

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
model = make_pipeline(StandardScaler(), LinearDiscriminantAnalysis(solver="lsqr",
shrinkage="auto"))
evaluate(model)
```

8. Code Explanation

`solver="lsqr"` supports covariance shrinkage, helpful when features are correlated or samples are limited. Scaling is not theoretically required but is often sensible for numeric stability and workflow consistency.

9. Training / Evaluation

Check Gaussian/shared-covariance plausibility with plots and validation. Use shrinkage when d approaches/exceeds sample size. Compare with logistic regression and QDA using stratified CV.

10. Complexity and Cost

Estimating/inverting covariance costs about $O(d^3)$; prediction is $O(Cd^2)$ naively. Memory is $O(d^2)$. Good for modest feature dimensions.

11. Common Use Cases

Small-sample scientific classification, face recognition (Fisherfaces), sensor classification, and interpretable Gaussian baselines.

12. Common Mistakes

Confusing LDA classifier with latent Dirichlet allocation, assuming scaling fixes non-Gaussian data, inverting singular covariance without shrinkage, and forgetting the shared-covariance assumption.

13. Edge Cases / Limitations

Different class covariance shapes create nonlinear true boundaries; singular covariance and heavy tails can destabilize estimates. It is sensitive to outliers.

14. Variations

- **Shrinkage LDA:** regularizes covariance; important in high dimensions.
- **Fisher LDA projection:** supervised dimensionality reduction; placement-important.
- **Regularized discriminant analysis:** interpolates LDA/QDA; advanced.

15. Related Topics

LDA and logistic regression often yield similar linear boundaries, but LDA models $P(\mathbf{x}|\mathbf{y})$ while logistic regression models $P(\mathbf{y}|\mathbf{x})$. QDA relaxes the common covariance constraint.

16. Interview Questions

Question	Answer
LDA assumption?	Gaussian classes with equal covariance matrices.
Why boundary linear?	Quadratic $\mathbf{x}^T \Sigma^{-1} \mathbf{x}$ terms cancel with shared covariance.
LDA vs logistic regression?	Generative Gaussian model versus discriminative conditional model.
LDA vs QDA?	LDA shares covariance; QDA estimates one per class.
What is pooled covariance?	Shared covariance estimated by combining within-class scatter.
Why shrinkage?	Stabilizes covariance inversion with few/high-dimensional samples.
Max projection components?	At most number of classes minus one.
What are priors?	Class frequencies/beliefs before observing $\mathbf{x}$.

Question	Answer
When can LDA win?	Data roughly meets assumptions and sample size is limited.
Key failure?	Unequal/non-Gaussian covariances and outliers.

17. Practice Tasks

Derive two-class boundary; visualize LDA projection on Iris; compare plain/shrinkage LDA; create unequal-covariance synthetic data; benchmark against logistic regression.

18. Project Ideas

- **Wine cultivar classifier:** LDA projection + classifier, UCI Wine; visually compelling.
- **EEG state classification:** shrinkage LDA, BCI dataset; small-sample scientific ML.
- **Face subspace demo:** Fisherfaces, ORL faces; classic CV portfolio piece.

19. Quick Revision

Key idea: Gaussian classes, same covariance, linear boundary. Formula: $\delta_k(\mathbf{x})$. Use: small Gaussian-like data/dimension reduction. Metrics: F1/AUC. Trap: LDA acronym confusion. One-liner: "A generative linear classifier from pooled covariance."

20. Final Cheat Sheet

Definition: shared-covariance Gaussian classifier. Input/output: vector $\rightarrow$ class posterior. Steps: means, pooled covariance, discriminant scores. Hyperparameters: solver/shrinkage/priors. Pros: fast/principled; cons: assumptions. Best: small scientific data.

Quadratic Discriminant Analysis

1. Overview

QDA is a generative Gaussian classifier that gives each class its own covariance matrix. It captures curved decision boundaries when classes have different spread or correlation structures.

2. Intuition

Instead of forcing all class ellipses to share one shape, give every class its own ellipse. Comparing differently shaped ellipses produces curved boundaries.

3. Prerequisites

LDA, multivariate Gaussians, covariance matrices/determinants, Bayes theorem, and regularization.

4. Core Concepts

- **Class covariance Σ_k :** models per-class shape; why: allows heteroscedastic classes. Interview: more flexible but many more parameters.

- **Quadratic boundary:** $x^T \Sigma_k^{-1} x$ terms no longer cancel.
- **Regularization:** blend covariance toward a stable diagonal/shared structure; vital with limited rows.

5. Algorithm / Working Process

Estimate class priors, mean, and covariance separately for each class. Score a test point under every class Gaussian and pick the largest posterior.

6. Mathematical Foundation

$\delta_k(x) = -\frac{1}{2} \log |\Sigma_k| - \frac{1}{2} (x - \mu_k)^T \Sigma_k^{-1} (x - \mu_k) + \log \pi_k$. Since Σ_k differs by class, the decision equation contains quadratic terms in x .

7. Practical Implementation

```
from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis
model = make_pipeline(StandardScaler(),
QuadraticDiscriminantAnalysis(reg_param=.1))
evaluate(model)
```

8. Code Explanation

`reg_param` shrinks individual covariance estimates toward a simpler estimate. Set it through cross-validation; warnings about collinearity/singularity are a signal, not noise to ignore.

9. Training / Evaluation

QDA needs enough samples **per class** to estimate $d(d+1)/2$ covariance values. Tune `reg_param`; compare validation behavior with LDA. Validate rare classes carefully.

10. Complexity and Cost

Stores/inverts one $d \times d$ covariance per class: roughly $O(d^3)$ fitting and $O(d^2)$ per prediction, memory $O(d^2)$.

11. Common Use Cases

Biomedical signals, image/sensor features, and small datasets where class variance structures visibly differ.

12. Common Mistakes

Using QDA with too few samples per class, no covariance regularization, confusing a flexible boundary with automatic better generalization, and overlooking outliers.

13. Edge Cases / Limitations

High-dimensional small data leads to singular/noisy covariance estimates. It overfits readily and is unsuitable if class distributions are far from Gaussian.

14. Variations

- **Regularized QDA:** covariance shrinkage; practical essential.
- **Regularized discriminant analysis:** continuum between LDA and QDA; advanced interview topic.
- **Diagonal QDA / Gaussian NB:** ignores within-class correlations; use when data is scarce.

15. Related Topics

QDA is LDA with separate covariances. Gaussian NB is like diagonal QDA (conditional independence). Kernel SVM also gives nonlinear boundaries but is discriminative.

16. Interview Questions

Question	Answer
QDA assumption?	Gaussian features per class with class-specific covariance.
Why quadratic boundary?	Covariance-dependent quadratic terms remain.
QDA vs LDA?	More flexibility/parameters versus shared covariance/linear boundary.
Why overfit?	Estimates a covariance matrix for every class.
What does reg_param do?	Shrinks covariance estimates for stability.
When prefer QDA?	Sufficient data and visibly unequal class spreads/correlations.
Parameter count issue?	Each class covariance has $d(d+1)/2$ entries.
QDA vs Gaussian NB?	QDA models correlations; NB uses diagonal covariance.
Need scaling?	Helpful numerically, but does not solve covariance assumptions.
What warning matters?	Singular covariance means insufficient/collinear data.

17. Practice Tasks

Generate unequal-covariance blobs; plot LDA/QDA boundaries; vary samples per class; tune regularization; compare QDA to Gaussian NB.

18. Project Ideas

- **Wearable activity classifier:** QDA, UCI HAR features; analyze covariance differences.
- **Quality-control sensor model:** class-specific variability, SECOM-like data; statistical modeling story.
- **LDA/QDA boundary visualizer:** Streamlit + synthetic data; excellent interview demonstration.

19. Quick Revision

Key idea: one Gaussian covariance per class. Formula: Gaussian discriminant with Σ_k . Use: different class spreads with adequate data. Metrics: F1/AUC. Trap: covariance overfit. One-liner: "LDA's flexible quadratic sibling."

20. Final Cheat Sheet

Definition: separate-covariance Gaussian classifier. Input/output: vector $\rightarrow$ posterior/class. Steps: per-class mean/covariance, compare likelihoods. Hyperparameters: `reg_param`, priors. Pros: curved boundaries; cons: parameter-hungry. Best: low-dimensional heteroscedastic data.

Gaussian Processes

1. Overview

A Gaussian Process (GP) is a Bayesian non-parametric distribution over functions. For classification it produces probabilistic nonlinear decision functions with principled uncertainty, particularly valuable on small expensive datasets.

2. Intuition

Rather than committing to one curve, place a probability distribution over plausible curves. Nearby points influence each other according to a kernel; far from data, uncertainty grows.

3. Prerequisites

Multivariate Gaussians, covariance kernels, Bayes rule, matrix algebra/Cholesky decomposition, logistic/probit likelihood, and uncertainty calibration.

4. Core Concepts

- **Kernel:** $k(x, x')$ expresses function similarity; why: determines smoothness/structure. Interview: kernel is covariance, not merely a trick.
- **Posterior uncertainty:** GP outputs mean and variance; valuable for active learning.
- **Non-Gaussian classification likelihood:** sigmoid/probit makes exact posterior intractable; approximation is needed.

5. Algorithm / Working Process

Choose kernel and mean prior; form covariance among training rows; combine latent-function prior with class likelihood; approximate posterior; for a query compute posterior latent distribution and convert it to class probability.

6. Mathematical Foundation

$f \sim GP(m(x), k(x, x'))$; finite values satisfy $f(X) \sim N(m, K)$. For classification, $P(y=1|f) = \sigma(f)$ (or probit). Unlike GP regression, the posterior is not Gaussian; Laplace/variational/EP approximations are used. RBF kernel is $\sigma_f^2 \exp(-||x-x'||^2/(2\ell^2))$.

7. Practical Implementation

```
from sklearn.gaussian_process import GaussianProcessClassifier
from sklearn.gaussian_process.kernels import RBF, ConstantKernel
kernel = ConstantKernel(1.0) * RBF(length_scale=1.0)
```

```
model = make_pipeline(StandardScaler(), GaussianProcessClassifier(kernel=kernel,  
random_state=42))  
evaluate(model)
```

8. Code Explanation

The RBF length scale controls how quickly similarity decays; optimization can adjust kernel parameters. Use this example on hundreds or a few thousand rows, not massive data.

9. Training / Evaluation

Standardize features, compare kernels via CV/marginal likelihood, and assess calibration/uncertainty—not only accuracy. Keep a held-out test set; uncertainty is only as good as kernel/model assumptions.

10. Complexity and Cost

Exact GP operations need $O(n^3)$ time and $O(n^2)$ memory; classification approximations add cost. CPU suffices for small n ; sparse/variational GPs are required at scale.

11. Common Use Cases

Scientific experiments, Bayesian optimization feasibility classification, robotics, geospatial classification, and active learning with costly labels.

12. Common Mistakes

Using a GP on huge datasets, treating variance as guaranteed real-world confidence, ignoring feature scaling, selecting kernel after looking at test results, and confusing GP classification with regression.

13. Edge Cases / Limitations

Scaling is the dominant limitation. Kernel misspecification produces misleading uncertainty; extrapolation behavior follows the prior and can be poor.

14. Variations

- **GP regression:** Gaussian likelihood, exact posterior; foundational.
- **Sparse/inducing-point GP:** approximate scale-up; research/industry important.
- **Multi-class GP:** one-vs-rest or coupled approximations; advanced.

15. Related Topics

RBF SVM and GPs both use kernels, but SVM returns a margin classifier while GP maintains a Bayesian distribution over functions. Bayesian neural networks pursue similar uncertainty goals.

16. Interview Questions

Question	Answer
What is a GP?	A distribution over functions where finite function values are jointly Gaussian.
What is a kernel?	A covariance function defining similarity and function prior.
Why uncertainty?	Posterior variance reflects limited evidence under the model.
GP classifier likelihood?	Usually logistic or probit on a latent function.
Why approximate inference?	Logistic/probit likelihood breaks Gaussian conjugacy.
Complexity?	About cubic time and quadratic memory in training rows.
GP vs SVM?	Bayesian uncertainty/function distribution versus maximum-margin solution.
RBF length scale?	Distance over which function values remain strongly correlated.
Why scale inputs?	Kernel distances depend on feature units.
How scale GPs?	Inducing points/variational sparse approximations.

17. Practice Tasks

Plot GP mean/uncertainty on 1D data; change RBF length scale; compare GP and RBF SVM; use uncertainty for active sample selection; test sparse-GP library on larger data.

18. Project Ideas

- **Lab experiment classifier:** GP + uncertainty, small materials/medical dataset; research-ready.
- **Active-learning simulator:** choose next label by GP uncertainty, synthetic moons; compelling demo.
- **Land-cover mapping:** GP on geospatial features, small regional dataset; uncertainty map output.

19. Quick Revision

Key idea: Bayesian distribution over classification functions. Formula: $f \sim \text{GP}(\mathbf{m}, \mathbf{k})$. Use: small costly-data uncertainty problems. Metrics: AUC, log loss, calibration. Trap: cubic scaling. One-liner: "A kernel Bayesian classifier with uncertainty."

20. Final Cheat Sheet

Definition: non-parametric Bayesian function model. Input/output: feature $\rightarrow$ class probability/uncertainty. Steps: kernel covariance, posterior approximation, predict. Hyperparameters: kernel/length scale. Pros: uncertainty; cons: $\mathcal{O}(n^3)$. Best: small scientific active-learning data.

Bayesian Classifiers

1. Overview

Bayesian classifiers predict classes by combining prior beliefs with evidence and represent uncertainty in parameters and/or predictions. The term includes Naive Bayes, Bayesian logistic regression, Bayesian networks, and Bayesian generative models.

2. Intuition

Start with a belief—say fraud is rare—then update it after seeing transaction evidence. Unlike a single fixed parameter estimate, Bayesian learning can retain uncertainty about what the model itself knows.

3. Prerequisites

Bayes theorem, priors/posteriors, likelihood, conditional independence, probability distributions, conjugacy, Monte Carlo/variational inference, and calibration.

4. Core Concepts

- **Prior, likelihood, posterior:** $p(\theta|D) \propto p(D|\theta)p(\theta)$; matters when data is limited. Interview: priors should be justified/sensitivity-tested.
- **Posterior predictive:** integrate parameters: $p(y^*|x^*, D) = \int p(y^*|x^*, \theta)p(\theta|D)d\theta$; captures parameter uncertainty.
- **Conjugacy/approximation:** conjugate models allow closed form; otherwise use MCMC, Laplace, or variational inference.
- **Bayesian network:** directed graph factorizing joint probability; interview: conditional-independence structure drives inference.

5. Algorithm / Working Process

Choose a likelihood and prior, observe labeled data, infer posterior over parameters/latent variables, then integrate/average predictions over posterior samples or an approximation. Output is a class posterior plus possibly credible uncertainty.

6. Mathematical Foundation

Bayes rule: $p(\theta|D) = p(D|\theta)p(\theta)/p(D)$. Classification uses posterior predictive above. MAP chooses $\text{argmax}_{\theta} p(D|\theta)p(\theta)$ and resembles regularized optimization; full Bayes averages rather than choosing one parameter point.

7. Practical Implementation

```
# A conjugate Bayesian classifier: Bernoulli Naive Bayes with smoothing.
from sklearn.naive_bayes import BernoulliNB
model = make_pipeline(StandardScaler(), BernoulliNB(alpha=1.0, binarize=0.0))
evaluate(model)
# For full Bayesian logistic regression, use PyMC/Stan and posterior samples.
```

8. Code Explanation

BernoulliNB is Bayesian because smoothing acts as a Beta-prior pseudo-count for binary features. It is a practical conjugate example; full Bayesian logistic regression needs sampling or variational inference, not just sklearn **LogisticRegression**.

9. Training / Evaluation

Perform posterior predictive checks, calibration analysis, and prior sensitivity checks. With MCMC, inspect convergence diagnostics (chains, effective sample size, R-hat). Split data before any feature/target transformations.

10. Complexity and Cost

Conjugate Naive Bayes is cheap. MCMC Bayesian classifiers can be expensive (many posterior samples); variational inference is faster but approximate. Memory scales with retained samples/parameters.

11. Common Use Cases

Medical diagnosis, fraud/risk with prior knowledge, uncertainty-aware decision support, causal/structured models, and low-data scientific systems.

12. Common Mistakes

Calling every probabilistic classifier Bayesian, choosing an arbitrary strong prior, reporting credible intervals as frequentist confidence intervals, ignoring MCMC convergence, and using posterior probability without calibration/decision costs.

13. Edge Cases / Limitations

Bad priors or wrong likelihood mislead results; inference can be slow and hard to operationalize. Bayesian uncertainty is model-conditional, not a guarantee under dataset shift.

14. Variations

- **Naive Bayes:** factorized generative classifier; essential placement topic.
- **Bayesian logistic regression:** prior over weights; useful uncertainty-aware linear model.
- **Bayesian networks:** structured conditional dependencies; important AI theory.
- **Bayesian neural networks:** distributions over weights; research-heavy and computationally hard.

15. Related Topics

Frequentist logistic regression estimates one best weight vector; Bayesian logistic regression retains a posterior over weights. Gaussian processes are Bayesian non-parametric classifiers. Calibration connects probabilities to real observed frequencies.

16. Interview Questions

Question	Answer
What is Bayes rule?	Posterior is likelihood times prior, normalized by evidence.

Question	Answer
What is a prior?	Belief about parameters/classes before current data.
MAP vs full Bayes?	MAP picks one posterior mode; full Bayes averages over posterior uncertainty.
Posterior predictive?	Prediction integrated over uncertain parameters.
Why use priors?	Stabilize low-data estimates and encode defensible knowledge.
What is conjugacy?	Prior and posterior have same distribution family, enabling closed form.
MCMC vs variational inference?	Sampling can be accurate but slow; VI is fast approximate optimization.
Is Naive Bayes Bayesian?	Yes: it applies Bayes rule with modeled priors/likelihoods.
What is a Bayesian network?	Directed acyclic graph encoding a joint-probability factorization.
Can Bayesian models be wrong?	Yes; uncertainty is conditional on prior, likelihood, and data assumptions.

17. Practice Tasks

Update a Beta-Bernoulli posterior by hand; compare MLE and MAP with few samples; build Naive Bayes spam filter; run posterior predictive calibration; fit Bayesian logistic regression in PyMC.

18. Project Ideas

- **Disease-risk decision support:** Bayesian network, UCI Heart Disease; demonstrates uncertainty and conditional dependencies.
- **Bayesian A/B classifier:** Beta-Binomial posteriors, marketing click data; excellent product-analytics project.
- **Uncertainty-aware churn:** Bayesian logistic regression, Telco Churn; compares credible intervals with standard model.

19. Quick Revision

Key idea: update beliefs using data and average over uncertainty. Formula: $\text{posterior} \propto \text{likelihood} \times \text{prior}$. Use: low-data/prior/uncertainty settings. Metrics: log loss, calibration, F1. Trap: unjustified priors/convergence. One-liner: "Probabilistic inference over parameters, not just point fitting."

20. Final Cheat Sheet

Definition: classifiers based on posterior probability. Input/output: evidence $\rightarrow$ class posterior/uncertainty. Steps: prior, likelihood, infer posterior, posterior predictive. Hyperparameters: priors/inference settings. Pros: uncertainty/knowledge; cons: assumptions/cost. Best: risk-sensitive low-data structured problems.